

Icon: A Framework for Measuring Information Flow

Framework Specification

JoonHa Park
Independent Researcher

May 12, 2026

Contents

Abstract	3
What Icon Is	4
Why Icon Matters	4
Background	5
How to Read This Document	5
Part 1 — Foundations	5
Section 1. The Five Measurements	5
Section 2. Precise Definitions	7
Section 3. Why These Five	8
Section 4. Why Noise Is Required	10
Section 5. The Two Companion Materials	11
Part 2 — Statistical Validity	12
Section 6. The Saturation Ceiling	12
Section 7. Trust τ — When to Trust a Measurement	13
Section 8. The Validity of ρ	14
Part 3 — Algorithm	14
Section 9. The Measurement Pipeline	14
Section 10. Aggregations	16
Section 11. Comparing Profiles Across Systems	16
Part 4 — PROTOCOL Discipline	17
Section 12. The Eight Categories	17
Section 13. The Manifest Schema	18
Section 14. PROTOCOL Versioning	19
Part 5 — Reference Implementation	19
Section 15. Code Structure	19
Section 16. Interface Contracts	20
Section 17. Public Surface	21
Part 6 — What Can Be Done With This	21
Read — Seeing What a System Does	22
Shape — Building or Changing a System Toward a Target	22
Test — Perturbing and Watching What Changes	23

How the Three Connect	24
Part 7 — Community Model	24
Section 18. Why the Linux Model	24
Section 19. Initial Seeding	25
Section 20. Contribution Guidelines	26
Part 8 — Limits and References	27
Section 21. Limitations	27
Section 22. References	28
Appendix A: Mathematical Definitions	30
A.0 How to Read This Appendix	30
A.1 Basic Track — Information from Scratch	30
A.2 Basic Track — From Information Theory to Icon	32
A.3 Bridge — Notation and Setup	34
A.4 The InfoNCE Estimator	35
A.5 Trust τ — Formal Classification	37
A.6 Closure Derivation — Formal	38
A.7 The η_t Identity	39
A.8 Sample Complexity	40
A.9 Aggregation Composition Rules	41
A.10 Identifiability	42
Appendix B: Using AI to Apply Icon	43
B.0 General Principles	43
B.1 Reading — Workflows for Seeing What a System Does	45
B.2 Shaping — Workflows for Building or Changing a System	45
B.3 Testing — Workflows for Perturbing and Watching What Changes	46
B.4 Closing	47
About This Specification	47

Abstract

Icon is a framework for measuring information flow through systems that produce internal representations as numerical tensors. It defines five measurements taken at each representation — F_{in} , F_{task} , F_{self} , F_{layer} , and ρ — a statistical check (Trust τ) that classifies each measurement as valid, saturated, or invalid, and a settings discipline (PROTOCOL) with manifest-based serialization that makes measurements reproducible across laboratories. Within the framework’s chosen variable set and noise-channel assumptions, the five measurements form a closed operational set

under enumeration: they are derived from the four canonical variables present at any representation-bearing layer, augmented with a noise channel that makes the otherwise-degenerate self-pair measurable. A reference implementation in Python is released under Apache 2.0 at <https://github.com/joonhai-official/icon> and archived on Zenodo at <https://doi.org/10.5281/zenodo.20184015>. This document specifies the framework; two companion materials provide its empirical anchor (Park, 2026a) and its conceptual motivation (Park, 2026b).

What Icon Is

Icon is a framework for measuring information flow through systems that produce internal representations as numerical tensors. It defines five measurements taken at each representation, a statistical check that tells you when to trust the measurements, and a settings discipline that makes measurements reproducible across laboratories.

The framework is small on purpose. Five measurements, one check, one discipline. Everything specific to a particular system — what architecture, what data, what task — is the user’s choice. Everything specific to a particular domain — what questions to ask of the measurements — is the community’s contribution.

Icon is released as open infrastructure under the Apache 2.0 license. The reference implementation, the measurement protocol, and the supporting materials are designed to be used, extended, replicated, and falsified by anyone. Throughout this document, *Icon* refers to the framework; the public repository and release use the stylized project name *ICON*.

Why Icon Matters

Modern AI is opaque in a specific sense. We know how to build large systems that produce internal representations. We have far fewer principled ways to measure what those representations are doing. This gap has practical consequences.

Science. Current research often compares systems by aggregate behavior — accuracy, loss, benchmark scores — while shared, reproducible measurements of internal information flow remain less standardized. Two architectures that achieve identical accuracy may be processing information in very different ways, and current practice lacks a common operational vocabulary for comparing such differences.

Cost. Architecture and chip design proceed largely by trial. A framework that measures where information concentrates, where bottlenecks form, and how much capacity is actually used could turn intuition into specification. If validated across scales, such measurements could inform architecture and hardware design — for example, capacity allocation, bottleneck identification, and architecture–hardware co-design.

Safety. Whether a system is trustworthy depends partly on whether its internal representations are doing what we think they are doing. Shared measurement protocols could

support the evaluation of safety claims, representational drift, and robustness interventions — for example, comparing defenses against attacks and detecting representational drift in deployed systems.

An open measurement substrate that any laboratory can inspect, run, and critique can become a candidate substrate for shared measurement if it proves useful under independent replication. Icon is a contribution toward that substrate. Its value depends on adoption, not on the author’s individual claims about it.

Background

Icon is developed alongside two companion materials by the present author: an empirical study of information capacity in neural networks (Park, 2026a) and a conceptual investigation of cross-substrate measurement (Park, 2026b). The three documents are released together as companion materials; Section 5 describes the two companions’ content. This document specifies the framework the three jointly establish — and stands on its own mathematical and operational foundations, so a reader interested only in the framework can proceed without consulting the companions.

How to Read This Document

The document is layered so different readers can enter at different points.

Want to start measuring immediately? Skim Part 5 (Reference Implementation) and follow the repository’s getting-started guide.

Want to understand the framework’s logic? Read Part 1 (Foundations) and Part 2 (Statistical Validity).

Want to extend the framework to a new domain? Read Parts 1–4, then Part 6 (What Can Be Done With This).

Want the formal mathematics? Appendix A provides precise definitions in two tracks: a basic track that builds from high-school mathematics, and an advanced track for readers who can start at the formal level directly.

Want to apply Icon with AI assistance? Appendix B provides workflows — organized as reading, shaping, and testing — for using AI tools to write the adapter, loader, and analysis scripts. It is a practitioner’s companion, not part of the framework specification.

Part 1 — Foundations

Section 1. The Five Measurements

A system that processes inputs produces internal representations along the way. A neural network maps an image through convolutional features, attention layers, projections.

A signal-processing pipeline maps a waveform through wavelet decomposition and feature extraction. A biological neural population transforms sensory input through cortical hierarchies. In every case, something enters, gets transformed, and passes forward through internal stages. The internal stages are representations.

Icon asks five questions at every representation. Together, the five answers form a profile — a fingerprint of what the representation is doing.

Question 1. How much of the input is encoded here? The input enters the system. By the time it reaches a particular representation, some of it has been preserved, some has been transformed, some has been discarded. We measure how much of the input’s information is encoded in this representation. We call this measurement F_{in} — input information density.

Question 2. How aligned is this representation with the task? Most systems are doing something — classifying, predicting, reconstructing. The task defines a target. We measure how much information the representation carries about that target. We call this F_{task} — task information density.

Question 3. How well does this representation survive its own noise channel? A useful representation should be robust. If we add small noise to it, the noisy version should still carry essentially the same information as the original. We measure how much information passes from the clean representation to its noisy version. We call this F_{self} — self-consistency under noise. F_{self} quantifies the channel capacity of the representation’s noise channel: how much of the representation’s information survives perturbation.

Question 4. How much passes from here to the next layer? Information flows through stages. Between any two consecutive representations there is a transmission. We measure how much of one representation’s information makes it to the next. We call this F_{layer} — inter-layer transmission. Drops in F_{layer} mark bottlenecks: places where the system loses information it had a moment ago.

Question 5. How spread out is the representation? A representation is a vector with multiple dimensions. Two representations can carry the same information content while distributing it very differently — one concentrating most variance on a single axis, another spreading it evenly. We measure this geometric spread separately from the information measurements. We call this ρ — representational dispersion. ρ ranges from “one dimension dominates” (close to 0) to “all dimensions equally used” (close to 1).

That is the entire measurement set. Five questions, five answers, one profile per representation.

The profile is the framework’s basic data unit. A single measurement produces one profile. A study produces many — across layers, across training epochs, across conditions. The next four sections give the precise definitions (Section 2), explain why these five are sufficient (Section 3), explain why the noise injection in Questions 1–4 is required (Section 4), and note the framework’s empirical origin briefly (Section 5).

Section 2. Precise Definitions

This section gives the formulas in their simplest form. Appendix A provides the full mathematical treatment, including how each quantity is estimated from finite data and including a basic track for readers who have not seen mutual information before.

Setup. Let X be the input, Y be the target, and Z_L be the representation at layer L , a vector of dimension d_L . Apply a small noise to Z_L :

$$\tilde{Z}_L = Z_L + \sigma \cdot \text{RMS}(Z_L) \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

where σ is a small number (reference value 0.1) and $\text{RMS}(Z_L)$ is the root-mean-square magnitude of the representation, used to scale the noise to the signal so that the channel is comparable across systems with different signal magnitudes. The noise is added during measurement only, never during training. Section 4 explains why the noise is required.

The five measurements.

$$F_{\text{in}}(L) = \frac{I(X; \tilde{Z}_L)}{d_L}$$

$$F_{\text{task}}(L) = \frac{I(Y; \tilde{Z}_L)}{d_L}$$

$$F_{\text{self}}(L) = \frac{I(Z_L; \tilde{Z}_L)}{d_L}$$

$$F_{\text{layer}}(L) = \frac{I(\tilde{Z}_L; \tilde{Z}_{L+1})}{d_{L+1}}$$

$$\rho(L) = \frac{\text{PR}(Z_L)}{d_L} \in \left[\frac{1}{d_L}, 1 \right]$$

Here $I(A; B)$ denotes the mutual information between random variables A and B — a standard quantity from information theory measuring how much knowing one tells you about the other. Appendix A.1 defines mutual information from scratch for readers new to the concept.

For ρ , the participation ratio is

$$\text{PR}(Z) = \frac{(\sum_i \lambda_i)^2}{\sum_i \lambda_i^2}$$

where λ_i are the eigenvalues of the covariance of Z . The participation ratio is a standard measure of effective dimensionality used in neural population analysis (Litwin-Kumar et al., 2017; Recanatesi et al., 2021). Intuitively, PR counts how many dimen-

sions are doing meaningful work: it equals 1 when all variance lies on a single axis (rank-one collapse), and equals d_L when variance is equally distributed across all dimensions. Dividing by d_L normalizes this count to the range $[1/d_L, 1]$ for cross-layer comparison.

Units and normalization. The four F measurements have units of nats per dimension. The division by d_L (or d_{L+1} for F_{layer}) makes them comparable across layers of different sizes. ρ is unitless.

The profile. The five values at a given layer and time form one profile:

$$P(L, t) = (F_{\text{in}}, F_{\text{task}}, F_{\text{self}}, F_{\text{layer}}, \rho).$$

One canonical ratio. Among combinations of the five measurements, one ratio has a structurally privileged form. Define

$$\eta_t = \frac{F_{\text{task}}}{F_{\text{in}}} = \frac{I(Y; \tilde{Z}_L)/d_L}{I(X; \tilde{Z}_L)/d_L} = \frac{I(Y; \tilde{Z}_L)}{I(X; \tilde{Z}_L)}.$$

The d_L in numerator and denominator cancel exactly — algebraically, not approximately. This means η_t is invariant under the per-dimension normalization, and its values are directly comparable across representations of different dimensions and even different architectures. The ratio answers a specific question: of all the input information that this representation carries, what fraction is aligned with the task?

The exact d_L -cancellation makes η_t the framework’s most theoretically clean quantity. Empirically (Park, 2026a), this ratio correlates with task performance across architectures in a way that the individual components do not.

The five measurements above — and the canonical ratio η_t — are everything the framework computes. Section 3 derives why these five (and no others within the framework’s scope); Section 4 explains why the noise is required; Section 5 describes the two companion materials.

Section 3. Why These Five

The compactness of the measurement set is a design property, not an accident. The five components are the output of an enumeration over the variables present at any layer of a representation-bearing system. This section shows the enumeration. One step of the argument — the introduction of a noise channel that turns the otherwise-degenerate self-pair into a finite measurement — is used here ahead of its full justification; Section 4 provides that justification independently.

The four canonical variables. At any layer L of any representation-bearing system, exactly four variables are present:

1. The input X
2. The target Y

3. The current representation Z_L
4. The next representation Z_{L+1}

These four are canonical in the sense that they exist by virtue of the system being a representation-bearing system with a task. Any system with internal representations and a defined task has these four. The framework’s measurements are constructed from these and nothing else.

Pairing the variables. Mutual information is defined between two variables. With four variables, there are $\binom{4}{2} = 6$ possible pairs. We consider each:

Pair	Interpretation	Status
$(X, X), (Y, Y)$	Entropy of a single variable	Data property — excluded
(X, Y)	Task complexity	Data property — excluded
(X, Z_L)	Input–representation alignment	F_{in}
(Y, Z_L)	Target–representation alignment	F_{task}
$(X, Z_{L+1}), (Y, Z_{L+1})$	Same as above, one layer ahead	Redundant
(Z_L, Z_{L+1})	Inter-layer transmission	F_{layer}

Three of the six pairs survive pruning: $F_{\text{in}}, F_{\text{task}}, F_{\text{layer}}$. The framework adds two more measurements that the enumeration above does not directly produce.

The self-pair under noise. The pair (Z_L, Z_L) in the table above is excluded as “entropy of a single variable.” Without noise this is correct: $I(Z_L; Z_L) = H(Z_L)$ is just the entropy of Z_L , which is infinite for a continuous variable and not a useful measurement.

However, once the noise channel is introduced (Section 4 explains why), the pair $(Z_L, \tilde{Z}_L)$ becomes a new pair: it is no longer self-identical because $\tilde{Z}_L = Z_L + \text{noise}$ is a distinct random variable from Z_L . The mutual information $I(Z_L; \tilde{Z}_L)$ is finite and measures the channel capacity of the noise channel — how much of Z_L survives the noise. This is F_{self} .

F_{self} is therefore not an arbitrary addition. It is the pair that the enumeration excluded without noise and that the noise channel makes available.

The geometric augmentation. Mutual information measures amounts of information. It does not measure how that information is distributed across dimensions. A representation with all variance on one axis has the same mutual information with the input as one with variance spread evenly across all axes, but the two are structurally different objects.

ρ is the minimal geometric measurement that captures this distinction. It is a single number characterizing the effective dimensionality of the representation. More elaborate geometric measurements are possible (skewness, kurtosis, manifold dimension, persistent homology), but each adds a parameter the user must specify and a quantity to interpret. ρ is the smallest geometric augmentation that gives the framework dimensional awareness; the framework leaves richer geometric analysis to extensions.

Closure. Within the four canonical variables, the noise-channel augmentation, and the minimal geometric augmentation, the five components are the complete set. Adding more is possible only by extending the variable set (for example, adding a perturbation variable δ for adversarial analysis), and any such extension is a defined change to the framework’s scope rather than an addition within scope.

The framework’s compactness is therefore not arbitrary smallness. It is the closure of an enumeration. Five measurements are sufficient because the variables present at any layer admit exactly five distinct, non-redundant pairings under the framework’s structural choices.

Section 4. Why Noise Is Required

A property of mutual information forces a design choice in any framework that measures information flow through deterministic systems. This section explains the property, the resulting problem, and the resolution.

The deterministic identity. If $Z = f(X)$ is a deterministic function of X , then

$$I(X; Z) = H(Z) \leq H(X).$$

The mutual information equals the entropy of Z , regardless of what the function f is. Two different transformations f_1 and f_2 that produce representations with the same entropy have the same mutual information with the input. The measurement does not distinguish the transformations; it only measures the entropy of the output.

For continuous representations the situation is worse: $H(Z)$ is formally infinite, and any finite estimate is an artifact of the estimator’s discretization.

The consequence. A framework that measures $I(X; Z)$ on deterministic representations measures the entropy of Z , not the information-processing role of f . Two architectures that produce equally-entropic representations with completely different internal structure return the same F_{in} . The measurement fails to characterize what we want to characterize.

The resolution: Goldfeld’s noise channel. Goldfeld et al. (2019) showed that adding a small isotropic Gaussian noise to the representation turns the problem into a noise-channel problem with finite, transformation-dependent mutual information. Define

$$\tilde{Z} = Z + \sigma \cdot \text{RMS}(Z) \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Now $I(X; \tilde{Z})$ depends on the geometry of Z relative to the noise scale. Representations that concentrate variance along axes orthogonal to noise-sensitive directions yield higher $I(X; \tilde{Z})$; representations that spread variance into noise-dominated directions yield lower $I(X; \tilde{Z})$. The mutual information now distinguishes transformations.

Why scale to RMS. The noise is scaled to $\text{RMS}(Z)$ so that the signal-to-noise ratio is comparable across systems. Without this scaling, two systems with the same intrinsic

information structure but different signal magnitudes would receive different measurements. Scaling to RMS is the cross-system standardization that lets the measurement be comparable.

Why σ is user-declared. The value of σ shapes the measurement’s sensitivity. Very small σ approaches the deterministic limit. Very large σ overwhelms the signal. The reference value $\sigma = 0.1$ — small enough that signal dominates noise at typical scales, large enough that the noise channel is non-degenerate — is appropriate for most cases, but the user declares the value in their PROTOCOL (Part 4) and can override it for their domain.

Why measurement only, never training. The noise is a measurement device, not part of the system. Adding noise during training changes what the system learns; adding noise during measurement extracts information about the system as it actually is. The framework requires the noise channel for measurement and prohibits it during training.

ρ is exempt. The participation ratio ρ does not face this problem. It measures the geometric spread of variance, which is well-defined and finite for any covariance matrix without noise. ρ is therefore computed on the un-noised representation Z directly.

The noise channel is the framework’s most consequential structural choice. It is what makes mutual information measure transformations rather than just measure entropies. Without it, the framework’s four F components would be uninformative on deterministic systems — which includes essentially every artificial neural network.

Section 5. The Two Companion Materials

This document is released together with two companion materials by the present author. This section describes their content briefly.

The empirical companion (Park, 2026a) measures information capacity across eight neural architectures on a standard image dataset, producing 2,241 measurement records under a pre-specified protocol. It uses two of the framework’s components — F_{in} and F_{task} in this document’s notation — and documents that the same measurement methodology applies coherently across architectures, with consistent scaling behavior, phase-transition-like collapse in residual-free architectures, an estimator-bias-free signal in the ratio $\eta_t = F_{\text{task}}/F_{\text{in}}$, and a proof-of-concept for inverse design within the measured grid. The cross-architectural coherence is the empirical anchor that motivates this framework’s abstraction and extension.

The conceptual companion (Park, 2026b) articulates information flow as a vocabulary that abstracts away from substrate-specific implementation details. The argument is that any system processing inputs into representations participates in a common structural situation, and that information flow — as a vocabulary — captures cross-substrate measurement possibilities that substrate-specific vocabularies cannot.

This document specifies the framework that the two companions jointly motivate. Three of its five components (F_{self} , F_{layer} , ρ) extend the empirical companion’s measurement set; they are derived structurally in Sections 3 and 4 and are framework-level

proposals whose cross-architectural behavior will be tested in follow-up empirical work. The framework itself is independent of both companions: it stands on its own mathematical and operational foundations.

Part 2 — Statistical Validity

Section 6. The Saturation Ceiling

Every measurement that uses InfoNCE has a structural upper bound that the user must understand before interpreting any result. The bound is a known limitation of variational mutual information estimators (Poole et al., 2019; Song & Ermon, 2020) and is the principal constraint Trust τ accounts for.

The bound. The InfoNCE estimator with batch size B satisfies

$$\hat{I}_{\text{NCE}}(A; B) \leq \log B.$$

This is a property of the estimator, not of the underlying mutual information. The true mutual information $I(A; B)$ may be much larger than $\log B$; the estimator cannot return a value above $\log B$ regardless. With the framework’s reference batch size $B = 512$,

$$F_{\max} = \frac{\log 512}{d_L} \approx \frac{6.238}{d_L} \quad \text{nats per dimension.}$$

Consequences.

Saturation. A measurement whose estimate is close to $F_{\max}$ may have a true value far above this ceiling. Two systems whose true MI both exceed $\log B$ produce indistinguishable measurements at the ceiling. The framework treats such measurements as uninterpretable comparisons: they are not zero, not noise, but they cannot distinguish the systems.

Permutation floor. Even when the variables A and B are independent — when the (A, B) pairs are shuffled before estimation — the estimator returns a positive value that depends on dimension and batch size. This is the estimator’s finite-sample bias floor, not a bug. The floor is real and must be measured for each configuration.

Interpretive consequence. A measurement near the ceiling and a measurement near the floor are both uninformative comparisons, but for opposite reasons. The framework distinguishes them with the next section’s classification.

The saturation ceiling is the single largest constraint on absolute interpretation of Icon measurements. Within fixed PROTOCOL (Part 4), relative comparisons remain meaningful because the ceiling is constant; across PROTOCOLs, calibration mapping is required.

Section 7. Trust τ — When to Trust a Measurement

The framework’s validity gate classifies each F component into one of three states based on its relationship to the saturation ceiling and the permutation floor. The classification is called Trust τ .

The three classifications.

A measurement is **saturated** if it is at or near the ceiling:

$$F \geq r_{\text{sat}} \cdot F_{\text{max}}, \quad r_{\text{sat}} = 0.95 \text{ (reference).}$$

A saturated measurement is at the estimator’s limit. The true value may be far above the measurement; comparisons among saturated measurements are uninformative.

A measurement is **invalid** if it is indistinguishable from the permutation floor:

$$|F - F_{\text{perm}}| < \varepsilon_{\text{sep}} \quad \text{or} \quad F < r_{\text{rel}} \cdot F_{\text{perm}}$$

with reference values $\varepsilon_{\text{sep}} = 0.005$ and $r_{\text{rel}} = 3.0$. An invalid measurement is dominated by estimator bias; treat it as noise.

A measurement is **valid** otherwise — specifically, if it is sufficiently above the floor and sufficiently below the ceiling:

$$F \geq r_{\text{abs}} \cdot F_{\text{max}}, \quad r_{\text{abs}} = 0.20 \text{ (reference),}$$

and it is neither saturated nor invalid. A valid measurement reflects the actual relationship between the variables.

Aggregate Trust τ of a profile. A profile has five components, four of which (F_{in} , F_{task} , F_{self} , F_{layer}) receive Trust τ classification. ρ is a deterministic geometric quantity, not an estimate, and is exempt from the classification (Section 8 discusses ρ ’s own validity criterion).

The aggregate rule is conservative:

- **invalid** if any component is invalid
- **saturated** if any component is saturated and none invalid
- **valid** otherwise (all four components valid)

Only profiles with all four F components valid are themselves valid. A single invalid component invalidates the profile.

Why this matters. The user does not have to choose whether to trust a profile — Trust τ chooses for them, by the criteria above, deterministically. The criteria are the same across architectures, datasets, and domains. Two laboratories analyzing the same system under the same PROTOCOL will arrive at the same Trust τ classifications.

Reference thresholds vs. user thresholds. The four threshold values (r_{sat} , ε_{sep} , r_{rel} , r_{abs}) are reference values, not framework-wide constants. Users who measure systems where the typical signal regime differs from the reference may declare their own thresholds in their PROTOCOL.

Reporting invalid profiles. The framework does not require that invalid profiles be discarded. They should be reported descriptively. An invalid profile contains information: it tells you that the estimator was not appropriate for that configuration, which is itself a finding.

Section 8. The Validity of ρ

ρ is exempt from Trust τ because it is not an estimate. It is a deterministic geometric quantity computed from the eigenvalues of the covariance of Z . However, finite-sample estimation of the covariance itself can be unreliable when the sample count is small relative to the dimension. The framework imposes a separate validity criterion for ρ .

Sample count requirement. For a representation of dimension d_L , the covariance estimate from N samples is reliable when

$$N \geq 4 \cdot d_L$$

with the reference factor 4 chosen so that the smallest eigenvalues of the empirical covariance are stable estimates of the population eigenvalues. Users who measure ρ with $N < 4d_L$ should report ρ with a low-sample flag in the manifest; users with $N \geq 4d_L$ report ρ as valid.

Numerical stability. The participation ratio computation involves a ratio of sums of eigenvalues. Eigenvalues below a numerical floor (reference: 10^{-12} times the largest eigenvalue) are clamped to that floor to avoid division-by-near-zero artifacts. The floor is part of the PROTOCOL.

ρ 's range. With $N \geq 4d_L$ and numerical floor applied, ρ takes values in $[1/d_L, 1]$ deterministically. There is no permutation null for ρ — shuffling samples does not change a covariance matrix's eigenvalues meaningfully — so the saturated/invalid/valid triplet does not apply.

The result is that ρ is the framework's most robust measurement: it has no estimator bias, no batch-size ceiling, and no permutation floor. Its single requirement is sufficient sample count, which is verifiable from the manifest.

Part 3 — Algorithm

Section 9. The Measurement Pipeline

A single Icon measurement runs the following pipeline. The pipeline is deterministic under fixed seeds and PROTOCOL. The reference implementation (Part 5) executes this pipeline; alternate implementations must produce identical results given identical inputs.

Inputs.

- An adapter wrapping the system under measurement.
- A data loader.
- A layer name.
- A PROTOCOL declaration.
- A master seed.

Pipeline.

Step 1 — Initialize seeds. The master seed is used to derive sub-seeds for data shuffling, noise injection, InfoNCE critic initialization, and permutation generation. Each sub-seed is derived deterministically.

Step 2 — Sample collection. The data loader produces N input–target pairs from the validation split. N is part of the PROTOCOL; reference value $N = \max(8B, 4d_L)$.

Step 3 — Forward pass with taps. The adapter executes a forward pass that exposes the activations at the requested layer. The result is $\{Z_L^{(n)}\}_{n=1}^N$. If F_{layer} is to be computed, $\{Z_{L+1}^{(n)}\}$ is collected on the same forward pass.

Step 4 — Noise injection. Compute $\text{RMS}(Z_L)$. Apply the noise:

$$\tilde{Z}_L^{(n)} = Z_L^{(n)} + \sigma \cdot \text{RMS}(Z_L) \cdot \varepsilon^{(n)}, \quad \varepsilon^{(n)} \sim \mathcal{N}(0, I_{d_L}).$$

For F_{self} , retain both the un-noised $Z_L^{(n)}$ and the noisy version $\tilde{Z}_L^{(n)}$.

Step 5 — InfoNCE estimation. For each of the four F components, run the InfoNCE estimator with the configured critic, batch size, and training steps:

- F_{in} : estimate $I(X; \tilde{Z}_L)$.
- F_{task} : estimate $I(Y; \tilde{Z}_L)$.
- F_{self} : estimate $I(Z_L; \tilde{Z}_L)$.
- F_{layer} : estimate $I(\tilde{Z}_L; \tilde{Z}_{L+1})$.

Then divide by d_L (or d_{L+1} for F_{layer}) to obtain the per-dimension density.

Step 6 — Permutation null estimation. For each F component, shuffle the pairs and re-estimate with the same critic and training steps. This produces F_{perm} for each component.

Step 7 — ρ computation. Compute the empirical covariance $\hat{\Sigma}$ of Z_L across the N samples. Diagonalize, apply the eigenvalue floor, compute $\text{PR}(\hat{\Sigma})$, divide by d_L to obtain ρ .

Step 8 — Trust τ classification. For each F component, compute $F_{\text{max}} = \log B/d_L$. Classify as saturated/invalid/valid by the criteria in Section 7. Aggregate to a profile-level classification.

Step 9 — Manifest generation. Record all PROTOCOL settings, all seeds, the sample count N , the layer dimension d_L , and timing information.

Section 10. Aggregations

A single profile is one (layer, time) snapshot. The framework defines four primitive aggregations that compose individual profiles into larger views.

Static. A single profile at a fixed layer and time: $\text{Static} = P(L_0, t_0)$.

Spatial. A sequence of profiles indexed by layer, at a fixed time:

$$\text{Spatial}(t_0) = \{P(L, t_0) : L \in \text{layers}\}.$$

Spatial aggregation answers questions about where information lives in the system at a given moment.

Temporal. A sequence of profiles indexed by time, at a fixed layer:

$$\text{Temporal}(L_0) = \{P(L_0, t) : t \in \text{times}\}.$$

Temporal aggregation answers questions about how information evolves.

Shift. The component-wise difference between two profiles:

$$\text{Shift}(P_b, P_a) = P_b - P_a.$$

Shift aggregation answers questions about what changed between two conditions.

Composition. The four primitives compose freely: Spatial-of-Shift, Temporal-of-Shift, Spatial $\times$ Temporal grids.

Statistical reduction. The framework recommends four reductions of a sequence: Mean for typical-case, Slope (regression coefficient) for trends, Range for variability, Correlation with an external quantity for relating profiles to outcomes. The user declares the reduction in their analysis; the framework provides the primitive.

Section 11. Comparing Profiles Across Systems

Within-PROTOCOL comparison. Two profiles measured under identical PROTOCOL are directly comparable. Component-wise differences and ratios are meaningful in absolute terms. This covers most use cases: comparing layers, checkpoints, conditions, or architectures evaluated by the same laboratory.

Cross-PROTOCOL comparison. Two profiles measured under different PROTOCOLs are not directly comparable. The estimator's ceiling depends on batch size; the floor depends on dimension and batch size; the noise scale shifts the entire measurement scale.

For cross-PROTOCOL comparison, the framework recommends two approaches:

Calibration via shared anchor. If both PROTOCOLs measure the same reference system, the difference between their measurements of the anchor system defines a calibration offset.

Comparison of η_t only. The ratio $\eta_t = F_{\text{task}}/F_{\text{in}}$ is mathematically invariant under per-dimension normalization and empirically less sensitive to PROTOCOL differences than the individual components.

Cross-laboratory comparison. Two laboratories measuring different systems under their own PROTOCOLs face cross-PROTOCOL comparison. The framework’s recommendation is that laboratories publish their full manifests so that calibration via shared anchors is possible. Over time, community accumulation of measurements creates the calibration anchors the framework needs.

Part 4 — PROTOCOL Discipline

Section 12. The Eight Categories

PROTOCOL is the framework’s settings declaration. Every measurement records a complete PROTOCOL; two measurements with the same PROTOCOL are directly comparable; measurements with different PROTOCOLs require calibration mapping.

The framework does not freeze numerical values. It freezes categories — the eight kinds of settings every measurement must declare. Specific values within each category are the user’s decision.

The eight categories.

1. Noise. Settings controlling the Goldfeld noise channel.

- σ (noise scale, reference: 0.1)
- RMS computation method (per-batch global scalar)
- RMS floor (minimum RMS value, reference: 10^{-6})

2. InfoNCE. Settings controlling the mutual information estimator.

- B (batch size, reference: 512)
- n_{steps} (critic training steps, reference: 2000)
- Critic architecture (separable cosine MLP with learnable scale)
- Critic hidden dimensions (reference: [256, 256])
- Optimizer (reference: Adam, learning rate 10^{-3})

3. Pool. Settings controlling sample collection.

- N (total sample count, reference: $\max(8B, 4d_L)$)
- Pool seed (data shuffling seed)
- Permutation count (reference: 1)

4. Trust τ . The four threshold values.

- r_{sat} (saturation ratio, reference: 0.95)
- ε_{sep} (separation minimum, reference: 0.005)
- r_{rel} (relative ratio, reference: 3.0)

- r_{abs} (absolute floor, reference: 0.20)

5. Numerical Safety.

- Eigenvalue floor for ρ (reference: 10^{-12} times the largest eigenvalue)
- Negative-eigenvalue tolerance (reference: -10^{-9})
- Numerical precision (reference: float32 for forward pass, float64 for covariance and eigenvalues)

6. Training (conditional). Used only when measuring during training of the underlying system.

- System optimizer, learning rate, batch size, epochs, checkpoint schedule.

7. Perturbation (conditional). Used only when measuring under perturbation.

- Perturbation type, magnitude, iterations.

8. Statistics.

- Bootstrap resamples (reference: 1000 when uncertainty intervals are required, 0 otherwise)
- FDR level (reference: 0.05 for multiple-comparison corrections)

Three universal requirements apply to every PROTOCOL:

- **Deterministic seeding.** Given the same master seed and the same PROTOCOL, repeated measurements must produce identical results on the same hardware.
- **Numerical stability.** Edge cases must have defined behavior, specified in Appendix A.
- **Manifest discipline.** Every measurement records the complete PROTOCOL plus all derived seeds and environmental context.

Section 13. The Manifest Schema

A manifest is the serialized record of one measurement. It contains everything needed to reproduce the measurement, modulo hardware-specific numerical variation.

Required fields. A manifest declares: framework and PROTOCOL versions; the system under measurement (adapter class, layer name and dimension); the data source (loader class, number of classes or unsupervised flag, sample count); the full PROTOCOL across all eight categories; the seed hierarchy (master, data, noise, critic, permutation); the environment (language version, framework, device, device count, deterministic-mode flag, UTC timestamp); and the results (the five components, the per-component permutation null and ceiling, the Trust τ classifications, and timing). The exact field layout for the reference implementation is documented in the repository.

Manifest hashing. The manifest’s PROTOCOL block alone defines comparability: two manifests with identical canonicalized PROTOCOL blocks are the same PROTO-

COL. The SHA-256 hash of the canonicalized PROTOCOL block is the PROTOCOL identifier.

Optional extensions. Implementations may add an extensions block for implementation-specific or domain-specific fields. The framework guarantees backward compatibility for the required fields; the extensions block has no compatibility guarantees.

Section 14. PROTOCOL Versioning

The framework evolves. New components may be added; new categories may be specified; existing reference values may be updated based on accumulated evidence. The framework manages this evolution through PROTOCOL versioning.

Version format. PROTOCOL versions use semantic versioning: MAJOR.MINOR.PATCH.

- **PATCH** — Reference value updates that do not change the measurement’s structure. Measurements made with the same MAJOR.MINOR version are comparable.
- **MINOR** — Additive changes: new categories, new optional fields, new measurements that extend the framework without breaking existing measurements.
- **MAJOR** — Breaking changes: removal of components, redefinition of measurements, structural changes to the manifest schema.

The current PROTOCOL schema version is 1.0; the framework’s first public release is v0.1.0. The framework commits to:

- No MAJOR version increase without at least 12 months of advance notice and a public migration guide.
- PATCH version updates published as additive changelog entries; older PATCH versions remain valid and comparable.
- MINOR version increases documented with the new fields, the changes’ rationale, and any new reference values.

Backward compatibility guarantee. A manifest written under any version v of the framework must remain readable and interpretable under any later version v' with the same MAJOR number. The reference implementation will maintain readers for all historical MINOR and PATCH versions within the current MAJOR.

Part 5 — Reference Implementation

Section 15. Code Structure

The reference implementation is released under the Apache 2.0 license and maintained in the framework’s repository at <https://github.com/joonhai->

official/icon. An archived release is available at <https://doi.org/10.5281/zenodo.20184015>. The Phase 0 empirical reproduction package — including raw JSONL measurement records and analysis scripts — is available at <https://github.com/joonhai-official/icon-empirical> and archived at <https://doi.org/10.5281/zenodo.20184000>. The package structure mirrors this specification, with each module corresponding to a section of the document: foundations and validity (Parts 1–2) live alongside the pipeline and aggregation primitives (Part 3) and the PROTOCOL machinery (Part 4). The full directory layout, dependency list, and examples are in the repository.

Design principles.

1. **Verifiability.** Every computation corresponds to a specification section. Module-level documentation maintains the mapping.
2. **Determinism.** All randomness is seeded; seeds are recorded in the manifest.
3. **Minimalism.** No domain-specific code in the kernel; those live in community packages.
4. **Stable interface.** The public surface (Section 17) is the framework’s contract with users.

Dependencies. The reference implementation depends on standard numerical libraries (linear algebra, statistics, a tensor framework for the InfoNCE critic) and a small serialization library. Users provide their own model framework; the adapter contract (Section 16) decouples the framework from any specific model framework.

Alternate-language implementations. The specification is language-agnostic. Implementations in other languages (Rust, Julia, Mojo, others) that satisfy the contracts in Section 16 and produce identical measurements under identical PROTOCOL and seeds are conforming implementations.

Section 16. Interface Contracts

A user of the framework implements two contracts.

The Adapter contract. An adapter wraps a system under measurement. It specifies three operations:

1. **Layer enumeration.** Return the ordered list of layer identifiers available for measurement. The order defines the spatial ordering used by Spatial aggregation. Adjacent identifiers define the layer pairs used by F_{layer} .
2. **Forward pass with taps.** Given an input batch and a list of requested layer identifiers, run a forward pass and return, for each requested layer, an activation tensor of shape $[N, d_L]$. Higher-rank activations (for example, feature maps from a convolutional layer) must be reduced to $[N, d_L]$ by the adapter; the reduction convention is the adapter’s choice and must be documented.
3. **Dimension lookup.** Given a layer identifier, return d_L — the post-reduction dimension of that layer.

The DataLoader contract. A data loader wraps a dataset for the measurement pipeline. It specifies three operations:

1. **Training batch.** Given a batch size and a seed, return one batch from the training split. The same (batch size, seed) pair must always return the same batch.
2. **Validation batch.** Given a batch size and a seed, return one batch from the validation split. Same determinism requirement.
3. **Class count.** Return the number of classes (for classification tasks) or a flag indicating an unsupervised setup.

Both contracts are minimal — three operations each. Anything more would couple the framework to specific model frameworks or data formats. Anything less would not give the pipeline what it needs.

Section 17. Public Surface

The public surface is the small set of operations that users interact with. Everything else is implementation detail.

Measurement. A single function takes an adapter, a data loader, a layer identifier, a PROTOCOL, and a master seed, and returns one profile.

Batch measurement. A convenience function takes the same arguments minus the layer identifier and returns one profile per layer (or per requested subset).

Aggregations. Functions corresponding to the four primitives in Section 10: static, spatial, temporal, and shift. They take profiles as input and return aggregated views.

PROTOCOL. A PROTOCOL declaration is a structured value that can be constructed with defaults, modified per-category, serialized to a manifest, restored from a manifest, hashed for identity, and compared to another PROTOCOL for compatibility.

Trust τ inspection. Helper functions for classifying individual components and aggregating the four component classifications into a profile-level classification.

Versioning. Constants for the framework version and the PROTOCOL version, and a helper for checking whether a manifest is compatible with the current framework.

The complete operation list, with signatures and examples, is in the repository.

Type stability. The public surface is the framework’s stable contract. Within a MAJOR version, operation signatures only gain optional parameters, return types remain compatible, and behavior remains specification-compliant. Internal modules carry no such guarantees.

Part 6 — What Can Be Done With This

The framework’s measurements admit three kinds of use. The use does not depend on what field the user comes from; it depends on what the user is trying to do with a

system. The three kinds are *reading* a system, *shaping* a system, and *testing* a system. Specific applications — in any field that studies systems with internal representations — descend from one of these three.

Read — Seeing What a System Does

The profile itself is the answer. A single measurement returns five numbers; many measurements return a structure. Reading asks where information lives, how it concentrates, how it spreads, and how it changes.

Activities that fall here:

- Characterizing where information concentrates and where it is absent across layers, and how the trajectory of η_t from input to output reveals where task selectivity emerges.
- Watching how the profile evolves during training, fine-tuning, or deployment — the temporal aggregation as a record of how a system’s information geometry comes into being.
- Comparing profiles across architectures, across pretraining objectives, across data variants, across model families — to see what differs beyond what aggregate behavior reveals.
- Locating where in a system a specific finding (a circuit, a feature, an attribution result, a probe response) lives, against a distribution-level baseline.
- Reading the system communication-theoretically: where the channel capacity is, where the codebook is utilized, where the cascade transmits, and where it bottlenecks. Substrates other than neural networks — signal-processing pipelines, biological neural populations, classical communication systems — admit the same reading.
- Characterizing a representation quantitatively rather than qualitatively — producing its fingerprint, not its description.

Whoever studies what a representation is doing — by whatever name the work is given in its field — reads.

Shape — Building or Changing a System Toward a Target

The profile is the target; the configuration is the means. Shaping asks how to build, what to keep, what to remove, where to allocate, and how to size.

Activities that fall here:

- Sizing layers to a target information density. Choosing width or depth from the desired η_t at the output, using a fitted forward model and its closed-form inversion.
- Identifying capacity that is over-provisioned (high allocated dimension d_L , low effective dimension $\rho \cdot d_L$) or under-provisioned (high F_{layer} relative to next-layer

width).

- Inverting from a target profile back to a configuration — the operational inversion that the framework’s empirical anchor (Park, 2026a) demonstrated for the architectural variables width and depth.
- Comparing two designs by their information signatures rather than only by their outputs — two systems with identical accuracy can be distinguished by the geometry of what they carry.
- Curating data so that what the system receives matches what the architecture can carry — evaluating augmentations, curricula, and dataset variants by their shift in η_t .
- Co-designing hardware to the information geometry the system actually produces, rather than to its allocated width — reducing area where effective dimension is small, increasing where transmission exceeds capacity.
- Allocating capacity where it is needed and removing it where it is not — sizing, pruning, distilling, compressing.

Whoever chooses what to build or change — by whatever name the work is given in its field — shapes.

Test — Perturbing and Watching What Changes

The shift between profiles is the answer. The framework does not specify what to perturb; it specifies how to read the response. Testing asks what is invariant, what is fragile, what propagates, and what is absorbed.

Activities that fall here:

- Comparing a clean profile to a perturbed one — under adversarial attack, distributional shift, intervention on the representation, ablation of a component, or any other intentional disturbance.
- Locating where in the system a perturbation propagates and where it is absorbed — F_{layer} shifts identify the propagation path; F_{self} shifts measure channel-level robustness at each representation.
- Watching a profile drift over time — establishing a baseline in deployment and alerting when $\|\Delta P\|$ exceeds a threshold. Drift as the temporal echo of cumulative perturbation.
- Characterizing what is invariant under the change and what is destroyed — component-wise reading of ΔP as the perturbation’s full signature.
- Comparing defenses, robustifying procedures, or training modifications by the shift suppression they achieve — a defense that keeps $\Delta P \approx 0$ under attack preserves the system’s information structure.
- Examining how a system behaves under conditions it was not trained for — testing as a way of asking what the system has actually learned, rather than what it has memorized.

Whoever asks what happens when conditions change — by whatever name the work is given in its field — tests.

How the Three Connect

Reading establishes what is there. Shaping changes what is there toward a goal. Testing probes how robust the change is. The three return to each other: every shape needs a read to verify it, every test needs a baseline read, every read suggests a shape or a test.

The activities listed under each are not exhaustive. They are common shapes of work the framework supports. A reader will find their own work fitting one of the three, or fitting between them, or combining several at once. The framework does not prescribe the field; it provides the substrate.

Whichever of the three the reader does, doing it shares the substrate with everyone else doing any of them. That shared substrate is what makes the framework’s social shape possible — taken up in Part 7.

Part 7 — Community Model

Section 18. Why the Linux Model

A measurement framework’s scientific value depends on adoption. A single laboratory’s measurements are a starting point; what makes them useful is comparison across laboratories — and that comparison requires a shared substrate.

Two approaches to building such a substrate are possible. One is centralized: a single team maintains the framework, validates extensions, and reviews contributions. This produces tight quality control but limits the framework’s growth to what the central team can review. The other is federated: a small invariant core is maintained centrally; everything else lives in community packages with their own maintainers. This loses central quality control over peripheral packages but allows the framework to grow as widely as the community contributes.

The Linux operating system is one example of the federated approach. The Linux kernel is small, stable, and centrally maintained; thousands of packages, drivers, and applications surround it under their own maintenance. The model has worked for three decades and supports infrastructure most of the world now depends on.

Icon draws inspiration from the Linux model. The framework specification (this document) and the reference implementation define the kernel — small, stable, centrally maintained. Domain extensions, application packages, and analysis pipelines live in community-maintained packages around the kernel.

Kernel responsibilities. The author, serving as initial maintainer, commits to:

- Maintaining the framework specification and the reference implementation.
- Coordinating PROTOCOL versioning (Section 14).

- Reviewing changes to the kernel interface contracts (Section 16) and the public surface (Section 17).
- Publishing the empirical anchors that ground the framework’s reference values.

Community responsibilities. The community contributes:

- Adapter packages for specific model frameworks.
- Data loader packages for specific dataset formats.
- Domain extension packages mapping the kernel to specific fields.
- Lookup table contributions — published (configuration → profile) measurements that enrich the framework’s empirical anchors.
- Replication, extension, falsification — all welcome.

The framework’s invitation is symmetric: the initial maintainer commits to a small, stable core; the community is invited to build on it.

Who this is for. This framework is open to whoever finds it useful. Anyone who measures with it, reproduces what others have measured, extends it to a new substrate, refines an existing reading, reinterprets its outputs through a different lens, or falsifies one of its claims is welcome to contribute. The framework does not ask where you come from; it asks what you do with it.

Measure. Reproduce. Extend. Refine. Reinterpret. Falsify. Read it from whatever vantage your work gives you. Each of these is a contribution.

Section 19. Initial Seeding

The Linux model requires initial adopters. A framework with no measurements, no extensions, and no community is not yet a federated ecosystem; it is a specification waiting to become one. This section names the author’s seeding commitments — what the framework will provide before community contributions arrive.

Already available.

- The reference implementation — implementing the framework specified in this document, with the two interface contracts (Section 16) and the public surface (Section 17). Released at <https://github.com/joonhai-official/icon> and archived at <https://doi.org/10.5281/zenodo.20184015>.
- The empirical baseline (Park, 2026a) — 2,241 measurement records across 8 architectures, with documented protocol and verified reproducibility. The records constitute the framework’s first (configuration → profile) lookup contributions. Released at <https://github.com/joonhai-official/icon-empirical> and archived at <https://doi.org/10.5281/zenodo.20184000>.

In active development.

- Follow-up empirical work — extending the empirical baseline to additional architectures, datasets, and the framework components (F_{self} , F_{layer} , ρ) that the

baseline did not measure. This validates the framework’s structural extensions (Section 3) against cross-architectural evidence.

Suggested initial packages. The framework’s ecosystem grows by accretion. Early packages might wrap an adapter for a class of systems, extend the PROTOCOL to a new modality, automate a common analysis pattern, or bridge the kernel to a domain’s existing vocabulary. The exact shape of contributions is for contributors to discover; the kernel’s stability is what makes the discovery possible.

The seeding is not the ecosystem. The author’s commitments establish the substrate; the framework becomes useful infrastructure only when other laboratories contribute. The invitation is open from the framework’s release: any laboratory using the kernel for its own measurements contributes to the substrate by publishing manifests; any domain expert writing an extension package contributes by codifying their domain’s mapping; any researcher attempting replication or falsification contributes by testing the framework against new ground.

The framework’s growth is not the author’s project alone. It is the community’s, from the point of release forward.

Section 20. Contribution Guidelines

The framework’s stability depends on the kernel staying small and the contract being respected. The contribution guidelines below are deliberately minimal — they specify what must be true of a contribution, not what should be.

For adapter and loader packages.

- The package implements the contracts in Section 16 — three operations on the adapter side, three on the data-loader side. No more, no less.
- The package documents its tap convention (where in the underlying system it attaches taps, how higher-rank activations are reduced to 2D) in its README.
- The package is released under a license compatible with Apache 2.0.

For domain extension packages.

- The package’s mapping of the framework’s measurements to the domain’s quantities is documented in its README, with explicit reference to the framework specification.
- Reference values that the package overrides from the framework defaults are documented and justified.
- The package does not modify the kernel or override the public surface; it builds on them.

For lookup table contributions.

- The contribution publishes complete manifests for all measurements (the schema in Section 13).

- The contribution declares its PROTOCOL hash (Section 13) so cross-contribution comparability can be verified.
- The contribution publishes the analysis scripts that reproduce its claims from the raw manifests.

For framework changes (kernel-level).

- Changes to the kernel specification, the public surface, or the interface contracts go through the initial maintainer.
- MAJOR version changes (breaking) require 12 months of advance notice (Section 14).
- MINOR version changes (additive) are released with a changelog and migration documentation.
- PATCH version changes (reference value updates) are released with the empirical evidence motivating the update.

The maintainer’s commitment to the community is the inverse: the kernel will not change silently, backward compatibility within a MAJOR version will be preserved, and contributors will not be required to track unpublished changes.

Part 8 — Limits and References

Section 21. Limitations

The framework is honest about what it does not do.

Estimator-bound. The InfoNCE estimator provides a lower bound on mutual information. Absolute values systematically underestimate true MI. Relative comparisons under fixed PROTOCOL preserve ordering because the bias is consistent in direction; absolute claims require explicit attention to the saturation ceiling (Section 6). The formal limitations of variational mutual information estimators in general — including impossibility results for distribution-free estimation in high dimensions — are characterized by McAllester and Stratos (2020) and Song and Ermon (2020).

Saturation ceiling. The estimator has a maximum value $\log B$ determined by batch size. Measurements approaching this ceiling are uninterpretable as comparisons (Section 6). Trust τ flags these explicitly; users must respect the flag.

Classification-typical F_{task} . F_{task} is currently specified for tasks with a defined target variable Y . Continuous targets (regression), structured targets (sequence-to-sequence), and multi-output targets are operationally measurable, but their encoding may require future PROTOCOL extensions. The framework’s other components (F_{in} , F_{self} , F_{layer} , ρ) apply to any task type.

Single-modality input. The reference PROTOCOL assumes a single-modality input X . Multi-modal inputs (image + text + audio) require future-version PROTOCOL extensions specifying how multi-modal inputs enter the estimator.

Tap convention. Where an adapter attaches taps to the underlying system matters. Different choices (before vs. after activation, before vs. after normalization, spatial-mean vs. flattened for CNN feature maps) give different profiles. The framework requires the adapter to document its convention; it does not enforce a single convention across all systems.

Bounded numerical reproducibility. Bit-exact reproduction across hardware is not guaranteed. Deterministic-seeded reproduction within a fixed environment is. The PROTOCOL records the environment; cross-hardware comparisons should expect small numerical variation within reported precision.

Empirical anchor coverage. The framework’s empirical anchor (Park, 2026a) measured F_{in} and F_{task} across 8 architectures. F_{self} , F_{layer} , and ρ are framework extensions whose cross-architectural behavior awaits validation in follow-up empirical work. Users should treat these three components as operationally defined and structurally motivated (Section 3), but not yet validated at the same empirical breadth as F_{in} and F_{task} .

Ecosystem dependence. The framework’s scientific value scales with community contribution. A framework with one laboratory’s measurements is a starting point; a framework with a hundred laboratories’ measurements is shared infrastructure. The framework cannot achieve the latter without the former laboratories contributing.

These are operational limits, not bugs. They constrain how the framework should be interpreted, not whether it works.

Section 22. References

Empirical and conceptual origin (this author).

- Park, J. (2026a). *Information Capacity in Neural Networks: An Empirical Study of κ Scaling*. Companion preprint. Reproduction package: <https://doi.org/10.5281/zenodo.20184000>.
- Park, J. (2026b). *Information Flow and Self-Reference Across Substrates: On Self-Reference, Substrate, and the Need for Measurement*. Companion preprint.

Information theory.

- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423.
- Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory* (2nd ed.). Wiley.

Mutual information estimation.

- Kraskov, A., Stögbauer, H., & Grassberger, P. (2004). Estimating mutual information. *Physical Review E*, 69(6), 066138.
- van den Oord, A., Li, Y., & Vinyals, O. (2018). Representation learning with contrastive predictive coding. arXiv:1807.03748.

- Belghazi, M. I., Baratin, A., Rajeswar, S., Ozair, S., Bengio, Y., Courville, A., & Hjelm, R. D. (2018). MINE: Mutual information neural estimation. *ICML 2018*, PMLR 80:531–540.
- Poole, B., Ozair, S., van den Oord, A., Alemi, A., & Tucker, G. (2019). On variational bounds of mutual information. *ICML 2019*, PMLR 97:5171–5180.
- McAllester, D., & Stratos, K. (2020). Formal limitations on the measurement of mutual information. *AISTATS 2020*, PMLR 108:875–884.
- Song, J., & Ermon, S. (2020). Understanding the limitations of variational mutual information estimators. *ICLR 2020*.

Goldfeld noise channel.

- Goldfeld, Z., van den Berg, E., Greenewald, K., Melnyk, I., Nguyen, N., Kingsbury, B., & Polyanskiy, Y. (2019). Estimating information flow in deep neural networks. *ICML 2019*, PMLR 97:2299–2308.

Effective dimensionality and participation ratio.

- Roy, O., & Vetterli, M. (2007). The effective rank: A measure of effective dimensionality. *EUSIPCO 2007*, 606–610.
- Litwin-Kumar, A., Harris, K. D., Axel, R., Sompolinsky, H., & Abbott, L. F. (2017). Optimal degrees of synaptic connectivity. *Neuron*, 93(5), 1153–1164.
- Recanatesi, S., Farrell, M., Lajoie, G., Deneve, S., Rigotti, M., & Shea-Brown, E. (2021). Predictive learning as a network mechanism for extracting low-dimensional latent space representations. *Nature Communications*, 12(1), 1417.

Information bottleneck and related approaches to information in neural networks.

- Tishby, N., Pereira, F. C., & Bialek, W. (1999). The information bottleneck method. *Proceedings of the 37th Annual Allerton Conference on Communication, Control, and Computing*, 368–377.
- Tishby, N., & Zaslavsky, N. (2015). Deep learning and the information bottleneck principle. *IEEE Information Theory Workshop (ITW)*.
- Shwartz-Ziv, R., & Tishby, N. (2017). Opening the black box of deep neural networks via information. arXiv:1703.00810.
- Saxe, A. M., Bansal, Y., Dapello, J., Advani, M., Kolchinsky, A., Tracey, B. D., & Cox, D. D. (2018). On the information bottleneck theory of deep learning. *ICLR 2018*.
- Achille, A., & Soatto, S. (2018). Emergence of invariance and disentanglement in deep representations. *Journal of Machine Learning Research*, 19(50), 1–34.
- Kolchinsky, A., Tracey, B. D., & Wolpert, D. H. (2019). Nonlinear information bottleneck. *Entropy*, 21(12), 1181.

Appendix A: Mathematical Definitions

A.0 How to Read This Appendix

This appendix has two reading tracks.

Basic track (A.1–A.3). For readers who have not seen mutual information or information theory before. The basic track builds from high-school mathematics — probability as relative frequency, logarithms as the inverse of exponentiation, expectation as a weighted average — and arrives at the framework’s definitions step by step. No prior information theory is assumed.

Advanced track (A.4–A.10). For readers with background in probability, information theory, and machine learning. The advanced track starts at the formal definitions, gives proofs of the framework’s structural properties, derives the InfoNCE estimator’s finite-batch behavior, and discusses identifiability and sample complexity.

Track recommendation.

Background	Where to start
High-school math only	A.1
Undergraduate calculus + probability	A.2
Graduate-level probability	A.3
Information theory (Shannon, MI)	A.4
Familiar with InfoNCE	A.5

The two tracks converge at A.4 (InfoNCE estimation) and continue together through the remainder.

A.1 Basic Track — Information from Scratch

Probability

A probability is a number between 0 and 1 that says how likely something is. If you flip a fair coin, the probability of heads is $P(\text{heads}) = 0.5$. If you roll a fair six-sided die, the probability of any specific face is $P(\text{face } i) = 1/6$.

A random variable is a quantity whose value depends on which outcome occurs. The result of one coin flip is a random variable X that takes the value “heads” or “tails.” The result of one die roll is a random variable Y that takes values 1 through 6.

The probability distribution of a random variable says how likely each value is. For the die, the distribution is $P(Y = 1) = P(Y = 2) = \dots = P(Y = 6) = 1/6$.

When a random variable takes continuous values (like a real number, not just integers), we describe its distribution with a density function $p(x)$: the probability that the variable’s value falls in a small interval $[x, x + dx]$ is approximately $p(x) dx$. The total probability over all possible values is 1.

Logarithms — the tool information theory uses

A logarithm answers the question: “what power do I raise the base to in order to get this number?” If we use base 2: $\log_2(8) = 3$ because $2^3 = 8$. If we use base $e \approx 2.718$ (the natural logarithm, written $\ln$ or just $\log$): $\log(e) = 1$ because $e^1 = e$.

The framework uses natural logarithms throughout. The unit of information is then called the nat. If you prefer base-2 logarithms, the unit is the bit, and $1 \text{ nat} \approx 1.44 \text{ bits}$.

Two properties of logarithms matter for information theory:

1. $\log(ab) = \log(a) + \log(b)$ — multiplying inside becomes adding outside.
2. $\log(1) = 0$ and $\log(x) \rightarrow -\infty$ as $x \rightarrow 0^+$ — the log of a small probability is a large negative number.

Surprise and Entropy

Information theory starts with a simple idea: a surprising outcome carries more information than an expected one. If you flip a coin and it comes up heads (probability 0.5), you have learned one fact among two possibilities. If you roll a die and it comes up 3 (probability 1/6), you have learned one fact among six possibilities — more information.

The mathematical measure of surprise is:

$$\text{surprise}(x) = -\log P(x).$$

The minus sign is there because $P(x) \leq 1$ makes $\log P(x) \leq 0$, and we want surprise to be a positive number. A more likely outcome (P closer to 1) has smaller surprise; a less likely outcome (P closer to 0) has larger surprise. An outcome with $P = 1$ has surprise 0. An outcome with $P \rightarrow 0$ has infinite surprise.

The entropy of a random variable is the average surprise across all its possible values, weighted by their probabilities:

$$H(X) = -\sum_x P(x) \log P(X = x).$$

For a continuous variable with density $p(x)$:

$$H(X) = -\int p(x) \log p(x) dx.$$

Entropy measures how uncertain a random variable is. A fair coin has the maximum entropy among coin distributions. A biased coin (99/1) has low entropy.

Joint and Conditional Information

When you have two random variables together, you can ask new questions. Let X be the input to a system and Y be the output. The joint distribution $P(X, Y)$ tells you the probability that both have specific values simultaneously. The joint entropy is

$$H(X, Y) = - \sum_{x,y} P(x, y) \log P(x, y).$$

The conditional entropy $H(Y | X)$ measures how much uncertainty about Y remains after you have learned X :

$$H(Y | X) = - \sum_{x,y} P(x, y) \log P(y | x).$$

If knowing X tells you nothing about Y , then $H(Y | X) = H(Y)$. If knowing X tells you Y exactly, then $H(Y | X) = 0$.

Mutual Information

Mutual information measures how much knowing one variable reduces uncertainty about another:

$$I(X; Y) = H(Y) - H(Y | X) = H(X) - H(X | Y).$$

The two equalities express symmetry. An equivalent formula uses the joint and marginal distributions:

$$I(X; Y) = \sum_{x,y} P(x, y) \log \frac{P(x, y)}{P(x)P(y)}.$$

Mutual information has three important properties:

1. **Non-negative.** $I(X; Y) \geq 0$ always.
2. **Zero iff independent.** $I(X; Y) = 0$ exactly when X and Y are independent.
3. **Bounded by entropy.** $I(X; Y) \leq \min(H(X), H(Y))$.

Mutual information is the basic quantity Icon measures. The framework's four F components — F_{in} , F_{task} , F_{self} , F_{layer} — are all mutual informations between specific pairs of variables, divided by a dimension.

A.2 Basic Track — From Information Theory to Icon

Why Mutual Information for Representations

When a system processes an input X into a representation Z , we want to ask: how much of the input is in the representation?

A natural answer is: $I(X; Z)$. This is mutual information between input and representation. If Z contains all the information about X , then $I(X; Z) = H(X)$. If Z contains none, then $I(X; Z) = 0$.

The same logic applies to the target Y : how much of the target is in the representation? Answer: $I(Y; Z)$.

These two — $I(X; Z)$ and $I(Y; Z)$ — are the basis of the framework’s F_{in} and F_{task} .

Why Divide by Dimension

A representation Z has multiple dimensions. A larger representation can carry more information than a smaller one, just by having more dimensions. If you want to compare per-dimension efficiency — how much information each dimension carries — you divide by the dimension count d_L :

$$F_{\text{in}}(L) = \frac{I(X; \tilde{Z}_L)}{d_L}.$$

The per-dimension normalization makes the measurement comparable across layers of different sizes and across architectures with different widths.

Why the Noise Channel

There is a subtle problem with measuring $I(X; Z)$ when Z is computed deterministically from X . If $Z = f(X)$ for any function f , then

$$I(X; Z) = H(Z).$$

The mutual information equals the entropy of Z , regardless of what the function f is. The measurement does not distinguish the functions; it only measures the entropy of their output.

The solution is to add a small amount of noise to the representation before measuring:

$$\tilde{Z} = Z + \sigma \cdot \text{RMS}(Z) \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Now $I(X; \tilde{Z})$ is not simply the entropy of $\tilde{Z}$; it depends on the geometry of how Z relates to X . Different functions f now produce different $I(X; \tilde{Z})$ values.

The Four F Components, Intuitively

- $F_{\text{in}} = I(X; \tilde{Z})/d$ — how much of the input survives in the noisy representation, per dimension.
- $F_{\text{task}} = I(Y; \tilde{Z})/d$ — how much of the target survives in the noisy representation, per dimension.
- $F_{\text{self}} = I(Z; \tilde{Z})/d$ — how much of the clean representation survives in the noisy version, per dimension. This is the capacity of the noise channel itself.
- $F_{\text{layer}} = I(\tilde{Z}_L; \tilde{Z}_{L+1})/d_{L+1}$ — how much of one layer’s information makes it to the next layer, per dimension of the next layer.

The Fifth Component, ρ

The four F components measure amounts of information. They do not measure how that information is spread across dimensions. Two representations can carry the same total information while looking very different.

The participation ratio captures this. For a representation Z whose covariance has eigenvalues $\lambda_1, \dots, \lambda_d$:

$$\text{PR}(Z) = \frac{(\lambda_1 + \dots + \lambda_d)^2}{\lambda_1^2 + \dots + \lambda_d^2}.$$

This counts how many dimensions are effectively in use. The framework normalizes:

$$\rho = \frac{\text{PR}(Z)}{d} \in \left[\frac{1}{d}, 1 \right].$$

ρ near $1/d$: extreme concentration. ρ near 1: uniform spread.

Summary of the Basic Track

Readers who have followed A.1 and A.2 can read the framework’s specification (Parts 1–8) and the formal Appendix A sections from A.3 onwards with full understanding.

A.3 Bridge — Notation and Setup

This section consolidates notation for the advanced track and serves as the bridge between basic and advanced readers.

Variables and Spaces

Symbol	Meaning
X	input random variable, in input space $\mathcal{X}$
Y	target random variable, in target space $\mathcal{Y}$
Z_L	representation at layer L , in $\mathbb{R}^{d_L}$
d_L	dimension of representation at layer L
$\tilde{Z}_L$	noisy version: $\tilde{Z}_L = Z_L + \sigma \cdot \text{RMS}(Z_L) \cdot \varepsilon$
ε	isotropic Gaussian: $\varepsilon \sim \mathcal{N}(0, I_{d_L})$
σ	user-declared noise scale (reference: 0.1)
$\text{RMS}(Z_L)$	$\sqrt{\mathbb{E}[\ Z_L\ ^2/d_L]}$
B	InfoNCE batch size (reference: 512)
N	total sample count (reference: $\max(8B, 4d_L)$)
$\lambda_i(Z)$	i -th eigenvalue of $\text{Cov}(Z)$

Mutual Information

$$I(A; B) = \mathbb{E}_{p(a,b)} \left[\log \frac{p(a,b)}{p(a)p(b)} \right] = H(A) - H(A \mid B).$$

The framework uses natural logarithms throughout; units are nats.

The Five Components, Formally

$$\begin{aligned} F_{\text{in}}(L) &= \frac{I(X; \tilde{Z}_L)}{d_L}, & F_{\text{task}}(L) &= \frac{I(Y; \tilde{Z}_L)}{d_L} \\ F_{\text{self}}(L) &= \frac{I(Z_L; \tilde{Z}_L)}{d_L}, & F_{\text{layer}}(L) &= \frac{I(\tilde{Z}_L; \tilde{Z}_{L+1})}{d_{L+1}} \\ \rho(L) &= \frac{\text{PR}(Z_L)}{d_L}, & \text{PR}(Z) &= \frac{(\sum_i \lambda_i)^2}{\sum_i \lambda_i^2}. \end{aligned}$$

The profile at layer L and time t :

$$P(L, t) = (F_{\text{in}}, F_{\text{task}}, F_{\text{self}}, F_{\text{layer}}, \rho).$$

The canonical ratio:

$$\eta_t = \frac{F_{\text{task}}}{F_{\text{in}}} = \frac{I(Y; \tilde{Z})}{I(X; \tilde{Z})}.$$

A.4 The InfoNCE Estimator

This section specifies the estimator the framework uses to compute mutual information from finite samples. InfoNCE was introduced by van den Oord, Li, and Vinyals (2018) in the context of contrastive predictive coding; its position within the broader class of variational mutual information lower bounds is analyzed by Poole et al. (2019). Alternative estimators include MINE (Belghazi et al., 2018) and the k-nearest-neighbor estimator of Kraskov et al. (2004); the framework uses InfoNCE for its bounded behavior under fixed batch size and for the ceiling/floor structure that Trust τ depends on.

Definition

For random variables $A \in \mathcal{A}$ and $B \in \mathcal{B}$ with joint distribution $p(a, b)$, and a learnable critic $f : \mathcal{A} \times \mathcal{B} \rightarrow \mathbb{R}_{>0}$:

$$\hat{I}_{\text{NCE}}(A; B) = \mathbb{E}_{(a,b) \sim p} \left[\log \frac{f(a, b)}{\frac{1}{K} \sum_{j=1}^K f(a_j, b)} \right]$$

where $\{a_j\}_{j=1}^K$ is a batch of K samples from the marginal $p(a)$.

Reference Critic

The framework’s reference critic is a separable cosine critic with learnable scale:

$$f(a, b) = \exp(s \cdot \cos(g(a), h(b)))$$

where $g : \mathcal{A} \rightarrow \mathbb{R}^k$ and $h : \mathcal{B} \rightarrow \mathbb{R}^k$ are small MLPs (reference: two hidden layers of size 256 each, ReLU activation, $k = 64$), and $s \geq 0$ is a learnable scalar. The critic is trained jointly with the measurement using Adam (reference: learning rate 10^{-3} , $n_{\text{steps}} = 2000$).

The Saturation Ceiling

The InfoNCE estimator satisfies a structural upper bound:

$$\hat{I}_{\text{NCE}}(A; B) \leq \log B$$

regardless of the true mutual information $I(A; B)$ or the critic.

Proof sketch. Let $f(a, b)$ be the critic value on the positive pair. The denominator includes $f(a, b)$ as one of its B terms. The ratio is

$$\frac{f(a, b)}{\frac{1}{B} \sum_j f(a_j, b)} = \frac{B \cdot f(a, b)}{\sum_j f(a_j, b)} \leq \frac{B \cdot f(a, b)}{f(a, b)} = B.$$

Taking log: $\hat{I}_{\text{NCE}} \leq \log B$.

Permutation Null

For independently sampled A and B , the true mutual information is zero: $I(A; B) = 0$. However, the InfoNCE estimator returns a positive value on independent variables — its finite-sample bias floor. The floor does not go to zero as the sample count grows; it remains a function of dimensions, batch size, critic, and training steps.

The framework records both the real-pairs estimate and the permuted-pairs estimate. Trust τ uses the difference to classify the measurement.

Asymptotic Behavior

In the limit $B \rightarrow \infty$, the InfoNCE estimator converges to a lower bound on the true mutual information:

$$\lim_{B \rightarrow \infty} \hat{I}_{\text{NCE}}(A; B) \leq I(A; B),$$

with equality when the critic is optimal (Poole et al., 2019). For finite B , the estimator is biased downward and bounded above by $\log B$.

A.5 Trust τ — Formal Classification

Component-Level Classification

For a single F component with measured value F , permutation null F_{perm} , and saturation ceiling $F_{\text{max}} = \log B/d$, define the classification function

$$\tau(F, F_{\text{perm}}, F_{\text{max}}; \theta) \in \{\text{valid}, \text{saturated}, \text{invalid}\}$$

parameterized by thresholds $\theta = (r_{\text{sat}}, \varepsilon_{\text{sep}}, r_{\text{rel}}, r_{\text{abs}})$.

The classification rules apply in order:

1. **Saturated** if $F \geq r_{\text{sat}} \cdot F_{\text{max}}$.
2. **Invalid** if not saturated and either $|F - F_{\text{perm}}| < \varepsilon_{\text{sep}}$ or $F < r_{\text{rel}} \cdot F_{\text{perm}}$.
3. **Valid** if not saturated, not invalid, and $F \geq r_{\text{abs}} \cdot F_{\text{max}}$.
4. **Invalid** (fallback) if none of the above.

Aggregate Classification

For a profile $P = (F_{\text{in}}, F_{\text{task}}, F_{\text{self}}, F_{\text{layer}}, \rho)$ with component classifications $\{\tau_i\}$ for the four F components:

- **invalid** if any $\tau_i = \text{invalid}$;
- **saturated** if no $\tau_i = \text{invalid}$ and at least one $\tau_i = \text{saturated}$;
- **valid** if all $\tau_i = \text{valid}$.

ρ does not contribute to the aggregate directly; ρ has its own validity (Section 8) based on sample count.

Reference Thresholds and Their Rationale

$$r_{\text{sat}} = 0.95, \quad \varepsilon_{\text{sep}} = 0.005, \quad r_{\text{rel}} = 3.0, \quad r_{\text{abs}} = 0.20.$$

These values were chosen with the following design principles:

- $r_{\text{sat}} = 0.95$: a measurement at or above 95% of the ceiling is in the regime where small changes in the underlying MI map to large changes in the measurement.
- $\varepsilon_{\text{sep}} = 0.005$: a measurement within 0.005 nats of the permutation null is within typical estimator noise.
- $r_{\text{rel}} = 3.0$: a measurement that is less than 3× the permutation null retains substantial floor contamination.
- $r_{\text{abs}} = 0.20$: a measurement below 20% of the ceiling is in the regime where estimator variance is large relative to signal.

These are reference defaults. The framework treats the thresholds as parameters; the choice is the user's, declared in PROTOCOL.

A.6 Closure Derivation — Formal

Section 3 gave the closure argument informally. This section provides the formal version.

The Variable Set

At any layer L of a representation-bearing system with input X , target Y , current representation Z_L , and next representation Z_{L+1} , the canonical variable set is

$$\mathcal{V}_L = \{X, Y, Z_L, Z_{L+1}\}.$$

Enumeration of Pairs

The set of unordered pairs from $\mathcal{V}_L$ has $\binom{4}{2} = 6$ elements:

$$\{(X, Y), (X, Z_L), (X, Z_{L+1}), (Y, Z_L), (Y, Z_{L+1}), (Z_L, Z_{L+1})\}.$$

Pruning

Rule 1: Exclude data-only pairs. A pair (A, B) is a data-only pair if neither A nor B is a representation. The only such pair in $\mathcal{V}_L$ is (X, Y) .

Rule 2: Identify redundant cross-layer pairs. The pairs (X, Z_{L+1}) and (Y, Z_{L+1}) are F_{in} and F_{task} evaluated at layer $L + 1$, not new measurements. We include each measurement only once per layer.

After pruning, the surviving pairs are $\{(X, Z_L), (Y, Z_L), (Z_L, Z_{L+1})\}$. These give F_{in} , F_{task} , and F_{layer} .

Augmentation 1 — The Self-Pair Under Noise

The pair (Z_L, Z_L) is excluded as a self-pair without noise. With the noise channel, the pair $(Z_L, \tilde{Z}_L)$ is not a self-pair: Z_L and $\tilde{Z}_L$ are distinct random variables. Their mutual information $I(Z_L; \tilde{Z}_L)$ is finite and is the Shannon channel capacity of the noise channel applied to Z_L . This gives F_{self} .

Augmentation 2 — Geometric Augmentation

The four pairs measure amounts of information. They do not measure how information is distributed across the dimensions of Z_L . The participation ratio is the minimal geometric augmentation that captures this with a single scalar parameter.

Closure

The five components are the output of:

1. Variable enumeration over $\mathcal{V}_L$;

2. Pruning by data-only and redundancy rules;
3. Augmentation by the noise channel (enabling the self-pair);
4. Augmentation by the minimal geometric measurement.

Within this scope, the five are the complete set. Adding components requires extending the variable set, which is a defined scope change through PROTOCOL versioning (Section 14).

A.7 The η_t Identity

Section 2 introduced $\eta_t = F_{\text{task}}/F_{\text{in}}$ and asserted that its d_L dependence cancels exactly. This section verifies the identity.

The Cancellation

By definition,

$$\eta_t = \frac{F_{\text{task}}}{F_{\text{in}}} = \frac{I(Y; \tilde{Z}_L)/d_L}{I(X; \tilde{Z}_L)/d_L}.$$

The d_L factors appear in both numerator and denominator and cancel exactly:

$$\eta_t = \frac{I(Y; \tilde{Z}_L)}{I(X; \tilde{Z}_L)}.$$

The identity is algebraic, not approximate, and holds for every measurement.

Why This Matters

A central concern with per-dimension normalization is that the normalization itself may dominate any pattern observed. The ratio η_t is immune to this concern. Whatever d_L -dependence exists in F_{in} is exactly cancelled by the same dependence in F_{task} . A correlation of η_t with task performance cannot be attributed to per-dimension normalization artifacts.

Empirical Robustness

The empirical baseline (Park, 2026a) verified this identity numerically: across $n = 633$ measurement records, computing η_t from $F_{\text{task}}/F_{\text{in}}$ versus computing η_t from raw $I(Y; \tilde{Z})/I(X; \tilde{Z})$ produced maximum absolute difference 6.15×10^{-7} — machine precision.

Information Bottleneck Connection

The ratio η_t is structurally related to the Information Bottleneck framework (Tishby, Pereira, & Bialek, 1999; Tishby & Zaslavsky, 2015), which defines a Lagrangian

$$\mathcal{L}_{\text{IB}} = I(Y; Z) - \beta \cdot I(X; Z)$$

balancing task information against input information. The framework’s η_t measures the ratio $I(Y; Z)/I(X; Z)$ without committing to a particular β . The framework does not require IB framing; η_t is a measurement, IB is one possible interpretation.

Subsequent work has examined how information bottleneck dynamics manifest in neural network training (Shwartz-Ziv & Tishby, 2017), contested the universality of the compression phase observed there (Saxe et al., 2018), formalized the link between IB and learned invariance (Achille & Soatto, 2018), and addressed estimator issues that earlier IB studies encountered (Kolchinsky, Tracey, & Wolpert, 2019). The framework is compatible with these readings but does not depend on any of them; η_t is defined operationally and is interpretable independently of whether IB dynamics in fact govern the system under measurement.

A.8 Sample Complexity

This section gives sample counts N at which each component is reliably estimated under reference PROTOCOL.

F_in and F_task

For mutual information between X (or Y) and $\tilde{Z}$ via InfoNCE with batch size B , the estimator’s variance scales approximately as $1/B$ per training step and is averaged over the n_{steps} training steps. The reference critic with $n_{\text{steps}} = 2000$ achieves variance below 1% of the measurement at

$$N \geq 8B.$$

This is the framework’s reference N . Smaller N produces measurements with 2–5% variance — usable but with explicit caveats.

F_self and F_layer

F_{self} and F_{layer} have the same estimator structure as F_{in} and F_{task} ; the same sample requirement applies: $N \geq 8B$.

A subtlety: F_{layer} requires both $\tilde{Z}_L$ and $\tilde{Z}_{L+1}$ to be collected on the same forward pass for the same input batch.

p

The participation ratio’s reliability depends on the eigenvalue spectrum of $\text{Cov}(Z)$, which requires

$$N \geq 4d_L$$

for the smallest eigenvalues to be stable.

Cross-Component Joint Requirement

When a full profile is measured at a single layer, the binding constraint is the larger of the two requirements:

$$N \geq \max(8B, 4d_L).$$

For $B = 512$ and $d_L \leq 1024$, the InfoNCE requirement dominates. For $d_L > 1024$, the ρ requirement dominates.

A.9 Aggregation Composition Rules

Section 10 introduced the four aggregation primitives. This section specifies the formal composition rules.

Type Signatures

Let $\mathbb{P}$ be the space of profiles. The four primitives are:

- **Static** : $\mathbb{P} \rightarrow \mathbb{P}$, $P \mapsto P$
- **Spatial** : $\mathbb{P}^k \rightarrow \mathbb{P}^k$ at fixed t
- **Temporal** : $\mathbb{P}^m \rightarrow \mathbb{P}^m$ at fixed L
- **Shift** : $\mathbb{P} \times \mathbb{P} \rightarrow \mathbb{P}$, $(P_b, P_a) \mapsto P_b - P_a$

Shift is component-wise subtraction in $\mathbb{P}$ (treating $\mathbb{P} \subset \mathbb{R}^5$).

Compositions

- **Spatial of Temporal**: $\mathbb{P}^{k \times m}$, a layer-by-time matrix.
- **Spatial of Shift**: $\mathbb{P}^k$, the shift evaluated across layers.
- **Temporal of Shift**: $\mathbb{P}^m$, the shift across time.
- **Spatial-Temporal of Shift**: $\mathbb{P}^{k \times m}$, full grid.

Statistical Reductions

The framework specifies four reductions of a sequence:

- **Mean**: $\bar{P} = \frac{1}{k} \sum_{i=1}^k P_i$
- **Slope**: $\hat{\beta} = \text{OLS}(P; \text{index})$
- **Range**: $\text{Range}(P) = \max_i P_i - \min_i P_i$
- **Correlation with external X** : $r(P, X)$

Trust τ Under Aggregation

Aggregation may combine profiles of different Trust τ classifications:

- **Spatial/Temporal:** the aggregate retains a Trust τ classification per profile in the sequence.
- **Shift:** ΔP 's Trust τ is the most conservative of P_a 's and P_b 's.

Trust τ propagation under aggregation is conservative by design: aggregated quantities are no more trustworthy than their inputs.

A.10 Identifiability

Profile Equivalence

Define two systems S_1, S_2 as profile-equivalent at layer L if their profiles satisfy

$$P_{S_1}(L) = P_{S_2}(L).$$

Profile-equivalent systems share, by construction:

- The same per-dimension input information density.
- The same per-dimension task information density.
- The same channel capacity under noise.
- The same forward transmission to the next layer.
- The same effective dimensionality.

What Profile Equivalence Does Not Imply

Profile equivalence does not imply:

- **Same architecture.** Different architectures can produce profile-equivalent representations.
- **Same outputs on individual inputs.** Information-theoretic equivalence is about distributions.
- **Same generalization behavior.** Profile equivalence at one layer does not constrain behavior outside that layer.
- **Same internal mechanism.** Two systems can route information differently while producing the same profile at any single layer.

Multi-Layer Identifiability

If two systems are profile-equivalent at every layer simultaneously, they share the entire layer-by-layer information signature, the full η_t trajectory, and the full pattern of bottlenecks and selectivities. This is a substantial constraint but still does not determine the system uniquely.

Practical Consequence

The framework's measurements characterize information-theoretic structure. They are sufficient for comparison among systems on information-theoretic grounds and

for tracking changes within a system on those grounds. They are not sufficient to determine a system up to architectural identity.

The framework is honest about this: profiles characterize one specific kind of equivalence class. Other questions about a system require other measurements that complement Icon’s information-theoretic ones.

Appendix B: Using AI to Apply Icon

B.0 General Principles

This appendix is a practitioner’s companion, not part of the framework specification. It describes how to use AI assistants (large language models) to accelerate the work of applying Icon to a new project. The framework itself is independent of any AI tool; this appendix exists because, in practice, AI tools substantially reduce the time required to write the adapter, the data loader, and the analysis scripts that surround a measurement.

The appendix is also a cautionary document. AI-generated code carries known risks — silent inaccuracy, surface fluency masking semantic errors, opacity under maintenance, and stylistic patterns that mark code as AI-generated to reviewers. The principles below address these risks directly.

Principle 1 — Verification Is the User’s Responsibility

AI tools generate code that looks correct. The user is responsible for verifying that the code is correct. The framework’s measurements are deterministic and well-specified; this means any error in the user’s adapter or loader will produce systematic deviations from the framework’s intended behavior, not random noise that averages out.

Verification has three components, in order of priority:

1. **Specification check.** Does the adapter implement exactly the three operations in Section 16? Does the loader implement exactly the three operations? AI-generated implementations sometimes add helpful-looking but unspecified methods. Remove anything not in the contract.
2. **Shape check.** Does the forward-pass operation return tensors with the documented shape $[N, d_L]$? AI-generated code sometimes returns higher-rank tensors without applying the reduction. Verify shapes by running the adapter on a small batch and printing the output shapes.
3. **Determinism check.** Does the loader return the same batch given the same (batch_size, seed)? Verify by calling the loader twice with the same seed and comparing.

The framework’s reproducibility guarantee depends on these three checks. None of them is automated by the framework; all are the user’s responsibility.

Principle 2 — Comments Are for Maintenance, Not for AI Disclosure

Code that uses AI assistance should be commented for future maintainers. Comments should explain what the code does and why — not how the AI was prompted. Comments that read as AI tool output (excessive hedging, redundant explanation of obvious operations, restating function names in prose) mark code as AI-generated and reduce reviewer trust.

The framework recommends a comment style that reads as if written by an experienced engineer:

- Comments explain non-obvious choices.
- Comments do not explain obvious operations.
- Comments are written in concise declarative English, not in the apologetic-explanatory register typical of generated text.
- Docstrings follow standard conventions without filler.

The objective is code that a reviewer would accept as written by a human engineer.

Principle 3 — Validation Across Three Levels

When AI is used to generate code, validation operates at three levels.

Level 1 — Unit tests. Each adapter operation and each loader operation should have a unit test that exercises it in isolation. The tests verify shape, determinism, and contract conformance.

Level 2 — End-to-end smoke test. After implementing the adapter and loader, run a measurement on a small layer with a small sample count. The output should be a profile with all five components present, Trust τ classifications computed, and a valid manifest.

Level 3 — Sanity comparison with the empirical baseline. If your system is similar to a system measured in the empirical baseline (Park, 2026a), your measurements should land in a similar range. Order-of-magnitude differences indicate a misconfiguration.

Principle 4 — Domain Expertise Cannot Be Outsourced

AI tools can write code. They cannot, reliably, decide where to tap representations, which target encoding is appropriate, what reference values the user’s domain requires, or what the measurement means in the user’s domain. These are domain-expert decisions. The user must make them; the AI tool can help implement them once decided.

Principle 5 — Document Your AI Use

Scientific work that uses AI assistance benefits from explicit disclosure. A short note in the project README or in the paper’s acknowledgments suffices.

B.1 Reading — Workflows for Seeing What a System Does

When this applies. You want to characterize what is happening inside a system — across layers, across training, across model variants, across data conditions. The deliverable is a profile or a set of profiles, with their Trust τ classifications and a clear comparison.

Workflow.

1. **Decide the comparison axis.** Spatial (across layers, fixed time)? Temporal (across time, fixed layer)? Cross-system (different architectures, same data)? The axis determines what your adapters and loaders need to expose.
2. **Adapter, possibly with checkpoint access.** Use an AI tool to draft an adapter that exposes the layers you want to read. For training dynamics, include checkpoint-loading so the same adapter wraps each saved state.
3. **Loader from a fixed split.** Hold the data constant across the comparison. The same validation batch at every measurement; no random sampling between conditions.
4. **Measurement campaign.** Use the batch measurement entry point to read every layer of every condition under a single PROTOCOL. Multiple seeds (3–5) per condition give variance estimates.
5. **Triangulation.** When static, temporal, and cross-condition views agree, the reading is robust. When they disagree, look for measurement error before interpreting the disagreement substantively.

Common pitfalls. Tap convention divergence across adapters; checkpoint state contamination (BatchNorm statistics, dropout state); inconsistent reduction of higher-rank activations.

B.2 Shaping — Workflows for Building or Changing a System

When this applies. You have a target — a profile, an η_t at a given layer, an effective dimension at a given stage — and you want to find a system that produces it, or modify a system toward it. The deliverable is a configuration, a design choice, or a modification supported by measurement.

Workflow.

1. **Specify the target.** What profile, or what aspect of a profile, defines success? Be explicit: target η_t at the output layer, target effective dimension $\rho \cdot d_L$ at a bottleneck, target F_{layer} across a stage.
2. **Forward model.** Fit a forward model from configurations to profiles. The empirical anchor (Park, 2026a) demonstrated that $\kappa(w, d) = C \cdot w^\alpha \cdot d^\beta$ achieves median 1.75% error in 5-fold cross-validation for the architectural variables width and depth.

3. **Inversion.** For the log-linear forward model: $\hat{w} = \exp((\log \kappa^* - \log C - \beta \log d)/\alpha)$. For nonlinear forward models, numerical inversion or constrained optimization.
4. **Verification.** Build the system at the predicted configuration. Measure it. Compare to the target. If the measured profile is within Trust τ tolerance of the target, the shape held. If not, the gap is itself a measurement that improves the forward model.
5. **Lookup table contribution.** Publish the (configuration $\rightarrow$ profile) pairs. Each contribution enriches the framework’s empirical anchors and lets future shaping start from a richer model.

Common pitfalls. Forward model overfitting; extrapolation outside the fit range; confusing allocated dimension d_L (architecture decision) with effective dimension $\rho \cdot d_L$ (measured outcome).

B.3 Testing — Workflows for Perturbing and Watching What Changes

When this applies. You want to characterize what happens to a system under a defined change — an adversarial attack, a distributional shift, an intervention on the representation, an ablation of a component. The deliverable is a shift signature: $\Delta P = P_{\text{perturbed}} - P_{\text{clean}}$ across the relevant layers and times.

Workflow.

1. **Define the perturbation.** Be explicit and complete: attack type and hyperparameters, distributional change and its specification, ablation target and procedure. The perturbation is part of the PROTOCOL.
2. **Adapter that supports perturbed forward passes.** Use an AI tool to draft an adapter wrapper that routes perturbed inputs through the same forward pass as clean inputs, so the only difference between the two measurements is the perturbation itself.
3. **Paired measurement.** Measure under both conditions with the same master seed and PROTOCOL. The same input must be measured clean and perturbed.
4. **Shift analysis.** Compute ΔP per layer. Component-wise reading: F_{self} shifts indicate channel-level fragility; F_{layer} shifts indicate where the perturbation propagates; spatial-of-shift across all layers gives the full signature.
5. **Threshold-based monitoring (when applicable).** For deployment monitoring, establish a baseline profile under clean conditions, measure periodically, alert when $\|\Delta P\|$ exceeds a threshold derived from the baseline’s natural variation.

Common pitfalls. Adversarial gradient leakage through the adapter; comparing perturbations measured under different PROTOCOLs; treating deployment drift as a single shift rather than a trajectory.

B.4 Closing

The framework’s mathematical content lives in the specification (Parts 1–8) and Appendix A. This appendix has translated that content into three workflows — reading, shaping, testing — each of which uses AI tools to accelerate the implementation work without compromising the framework’s reproducibility and verification standards. Specific applications descend from these three; the workflows are templates, not prescriptions.

The principles are constant across the workflows:

- Verification is the user’s responsibility. AI tools generate; users verify.
- Domain expertise cannot be outsourced. AI implements; users decide.
- Determinism and reproducibility are non-negotiable. The framework’s manifests, seeds, and PROTOCOL discipline make AI-generated measurements scientifically usable.

The appendix will evolve as the community contributes domain extensions.

About This Specification

This specification is released together with two companion materials by the present author: Park (2026a) on empirical information-capacity scaling and Park (2026b) on cross-substrate measurement as a conceptual project. Together, the three documents establish the framework’s empirical anchor, its conceptual motivation, and its operational specification respectively.

The specification’s claims rest on the empirical companion for the components measured there (F_{in} , F_{task} , the ratio η_t) and on structural derivation for the components proposed but not yet validated at the same scope (F_{self} , F_{layer} , ρ). Replication, criticism, extension, and falsification are welcomed.

End of Icon Framework Specification, release v0.1.0 (PROTOCOL schema 1.0).